

Oriëntatie op AI

Les 2: algoritmiek - zoeken

2025-2026 Semester 1

HBO-ICT



Agenda

1. Algoritmes en pseudocode
2. Zoeken
 1. Lineair zoeken
 2. Binair zoeken
3. Complexiteit



| Algoritmes en pseudocode

Algoritme

- Wat is een algoritme?

“Een eindige reeks instructies die vanuit een gegeven begintoestand naar een beoogd doel leidt.”

“Any well-defined computational procedure that takes some value, or set of values, as input and produces some value, or set of values, as output.”



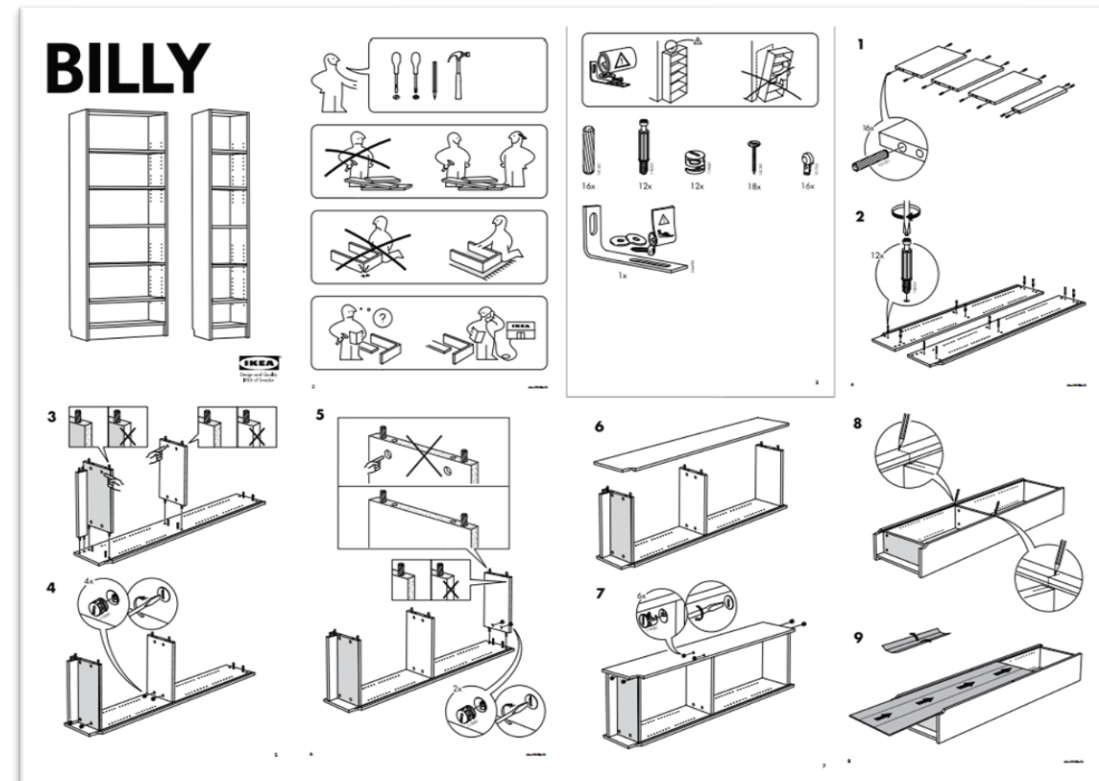
Algoritme

- Wat is een algoritme?
 - Welke algoritmes gebruik je in het dagelijks leven? Zou je hier een computerprogramma van kunnen schrijven?
 - Welke algoritmes worden gebruikt door de apps en games die je gebruikt?
 - Heb je ooit een algoritme gecodeerd? Wat deed het en was het algoritme effectief en efficiënt?

Algoritme

“Een eindige reeks instructies die vanuit een gegeven begintoestand naar een beoogd doel leidt.”

- Voorbeelden
 - Een recept voor een appeltaart
 - Mijn gewoontes bij het opstaan
 - Het in elkaar zetten van een Billy IKEA-kast
- Wat is de ‘begintoestand’ en wat is het ‘beoogde doel’?
- Kan een computer hiermee omgaan?



Algoritme

- Voorwaarden
 1. Volgordelijkheid
 2. Ondubbelzinnig
 3. Uit te rekenen
 4. Eindig
- Oefening
 - Zoek een IKEA instructie en een recept voor appeltaart – aan welke van de bovengenoemde voorwaarden voldoen deze instructies?

Pseudocode

- Een compromis tussen natuurlijke taal en programmeertaal
 - Natuurlijke taal (Nederlands, Engels) is te rijk, dubbelzinnig en contextafhankelijk
 - Programmeertaal is te concreet, teveel regels (grammatica, syntax)
- Pseudocode staat toe om op een abstracte manier over een probleem na te denken

Pseudocode

Veel algoritmen worden beschreven in **pseudocode**.

Algorithm 1 Deep Q-learning with Experience Replay

```
Initialize replay memory  $\mathcal{D}$  to capacity  $N$ 
Initialize action-value function  $Q$  with random weights
for episode = 1,  $M$  do
  Initialise sequence  $s_1 = \{x_1\}$  and preprocessed sequenced  $\phi_1 = \phi(s_1)$ 
  for  $t = 1, T$  do
    With probability  $\epsilon$  select a random action  $a_t$ 
    otherwise select  $a_t = \max_a Q^*(\phi(s_t), a; \theta)$ 
    Execute action  $a_t$  in emulator and observe reward  $r_t$  and image  $x_{t+1}$ 
    Set  $s_{t+1} = s_t, a_t, x_{t+1}$  and preprocess  $\phi_{t+1} = \phi(s_{t+1})$ 
    Store transition  $(\phi_t, a_t, r_t, \phi_{t+1})$  in  $\mathcal{D}$ 
    Sample random minibatch of transitions  $(\phi_j, a_j, r_j, \phi_{j+1})$  from  $\mathcal{D}$ 
    Set  $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$ 
    Perform a gradient descent step on  $(y_j - Q(\phi_j, a_j; \theta))^2$  according to equation 3
  end for
end for
```

Wat zie je allemaal?

Pseudocode

“Any well-defined computational procedure that takes some value, or set of values, as *input* and produces some value, or set of values, as *output*.”

Laten we eens wat eenvoudigers bekijken: het gemiddelde.

- Probleem: gegeven twee getallen, bereken het gemiddelde
- Algoritme:
 1. ontvang getallen `a` en `b`
 2. bereken `som = a + b`
 3. bereken `gemiddelde = som / 2`
 4. geef `gemiddelde` terug

Pseudocode

“Any well-defined computational procedure that takes some value, or set of values, as *input* and produces some value, or set of values, as *output*.”

- Probleem: verwijder alle klinkers uit een stuk tekst.
- Algoritme:
 1. ontvang tekst t
 2. herhaal stap 3. voor elke letter l in t
 3. als l geen klinker is, voeg deze toe aan de nieuwe tekst n
 4. geef n terug

Pseudocode

“Any well-defined computational procedure that takes some value, or set of values, as *input* and produces some value, or set of values, as *output*.”

- Probleem: gegeven twee getallen, bereken de grootste gemene deler
- Algoritme (van Euclides):
 1. ontvang twee positieve getallen
 2. noem het grootste getal a en het kleinste getal b
 3. herhaal stap 4 zolang a groter of gelijk is aan b
 4. trek b af van a
 5. als a gelijk is aan 0, geef dan b terug
 6. als a ongelijk is aan 0, ga naar stap 1 met b en (het restant van) a

Pseudocode

“Any well-defined computational procedure that takes some value, or set of values, as *input* and produces some value, or set of values, as *output*.”

- Probleem: gegeven twee getallen, bereken de grootste gemene deler
- Algoritme (Euclides):
 - gegeven getallen A en B
 - zolang ($A \neq B$)
 - als $A > B$
 - verminder A met B
 - anders
 - verminder B met A
 - retourneer A

Pseudocode – elementen

- Basale operaties
 - Krijg invoer
 - Geef uitvoer
 - Wiskundige bewerkingen (+, ×, etc.)
- Conditionele operaties
 - Alleen uitvoeren onder bepaalde condities
 - Bv. als ... dan ... anders ..., `if ... then ... else ...`
- Iteratieve operaties
 - Meerdere keren uitvoeren
 - B.v. herhaal, zolang, `while`, `for`

Pseudocode – oefening

- Schrijf een algoritme voor je ochtendritueel
 - Maak gebruik van een *conditie* en een *iteratie*
- Schrijf een algoritme om het gemiddelde uit te rekenen van een *willekeurig* aantal getallen



HOGESCHOOL
UTRECHT

Zoeken

Zoeken

- Lineair zoeken vs. binair zoeken
- Denk aan een (old school) telefoonboek
 - Zoek de naam bij een gegeven telefoonnummer
 - Zoek het telefoonnummer bij een gegeven naam

Zoeken – activiteit

- Raad het getal dat ik in gedachten heb:

22	35	39	15	27	12	32	1	3	38	2	25	8	33	30	40	7	26	24	6	14	21	31	5	18
1	2	3	5	6	7	8	12	14	15	18	21	22	24	25	26	27	30	31	32	33	35	38	39	40

Zoeken – animatie

<https://www.cs.usfca.edu/~galles/visualization/Search.html>

Searching Sorted List

411 ☒ Small ☐ Large

```
def binarySearch(listData, value)
    low = 0
    high = len(listData) - 1
    while (low <= high)
        mid = (low + high) / 2
        if (listData[mid] == value):
            return mid
        elif (listData[mid] < value):
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

Searching For 411 Result

low 0 mid 7 high 14

36	47	63	94	142	174	223	299	326	369	411	503	534	545	562	639
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

644	652	739	748	759	784	803	853	855	871	921	925	954	958	961	994
16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31

Animation Paused

Animation Speed

w: 1000 h: 500

Lineair zoeken – oefening

- Lineair zoeken: controleer of element `target` bestaat in lijst `lst`
 - Schrijf pseudocode voor het lineair zoekalgoritme

Lineair zoeken – oefening

- Lineair zoeken: controleer of element `target` bestaat in lijst `lst`
 - Schrijf pseudocode voor het lineair zoekalgoritme
 - Implementeer de functie `linear_search(lst, target)` in Python - Oefening AI2.1: Lineair zoeken (zie Canvas):
 - De functie retourneert **True** als `target` bestaat in `lst`
 - De functie retourneert **False** als `target` niet bestaat in `lst`

Binair zoeken

- Probleem: controleer of `target` bestaat in gesorteerde lijst `lst`
- Algoritme:
 1. Zet `mini = 0` en `maxi = lengte van lst - 1`
 2. Zet `index` op het gemiddelde van `mini` en `maxi`
 3. Als element `e` op positie `index` van lijst `lst` gelijk is aan `target`, retourneer succes – `target` is gevonden!
 4. Als element `e` op positie `index` van lijst `lst` kleiner is dan `target`, dan zet `mini = index + 1`
 5. Als element `e` op positie `index` van lijst `lst` groter is dan `target`, dan zet `maxi = index - 1`
 6. Ga naar stap 2 zolang `mini <= maxi`.

Zoeken – aantal stappen

- Maximaal aantal benodigde stappen om het gezochte element te vinden

Aantal elementen	Lineair zoeken	Binair zoeken
1	1	1
2	2	2
3	3	2
4	4	3
5	5	3
6	6	3
7	7	3
8	8	4
9	9	4
10	10	4
n	n	$1 + \lceil \log_2(n) \rceil$

| Complexiteit

Complexiteit van algoritmes

- Algoritmen hebben verschillende complexiteit
- De belangrijkste is *tijdscomplexiteit*: hoe lang duurt het om een probleem op te lossen (de looptijd)
- Daarnaast is er *ruimtecomplexiteit*: hoeveel ruimte (geheugen) is er nodig om een probleem op te lossen
- Hierbij kijken we *primair* naar hoe hard deze tijdsduur groeit naarmate het probleem groter wordt; in ons voorbeeld: als het aantal elementen n groeit
 - (We kijken dus *niet* naar bijvoorbeeld de snelheid van de computer)

Tijdcomplexiteit van algoritmes – scenario's

- **Worst-case scenario:** in welk scenario zijn de *meeste* stappen nodig om het probleem op te lossen, met andere woorden: wanneer is de looptijd het langst?
 - Bijv. bij lineair zoeken op 5 elementen: 5 stappen
- **Best-case scenario:** in welk scenario zijn de *minste* stappen nodig om het probleem op te lossen, met andere woorden: wanneer is de looptijd het kortst?
 - Bijv. bij lineair zoeken op 5 elementen: 1 stap
- **Average-case scenario:** wat is de *gemiddelde* looptijd?
 - Bijv. bij lineair zoeken op 5 elementen: 3 stappen

Complexiteit van algoritmes

- Algoritmen zijn in te delen in verschillende *klassen* van complexiteit:
 - klasse 1: constant (bijv. zoeken naar het 1^e element uit een lijst)
 - klasse $\log(n)$: logaritmisch (bijv. binair zoeken)
 - klasse n : lineair (bijv. lineair zoeken)
 - klasse n^2 : kwadratisch (bijv. zoeken naar dubbele elementen in een lijst)
 - klasse 2^n : exponentieel (bijv. oplossen van de Torens van Hanoi)
 - klasse $n!$: faculteit (bijv. het handelsreizigersprobleem)

Complexiteit van algoritmes



- Is complexiteit nog relevant met de computers van nu?
- Handelsreizigersprobleem

“Gegeven n steden samen met de afstand tussen ieder paar van deze steden, vind dan de kortste weg die precies één keer langs iedere stad komt.”
- Bij n steden zijn er $(n-1)!$ mogelijke routes
 - Bij 5 steden zijn dat 24 routes
 - Bij 10 steden zijn dat 362.880 routes
 - Bij 15 steden zijn dat 87.178.291.200 routes

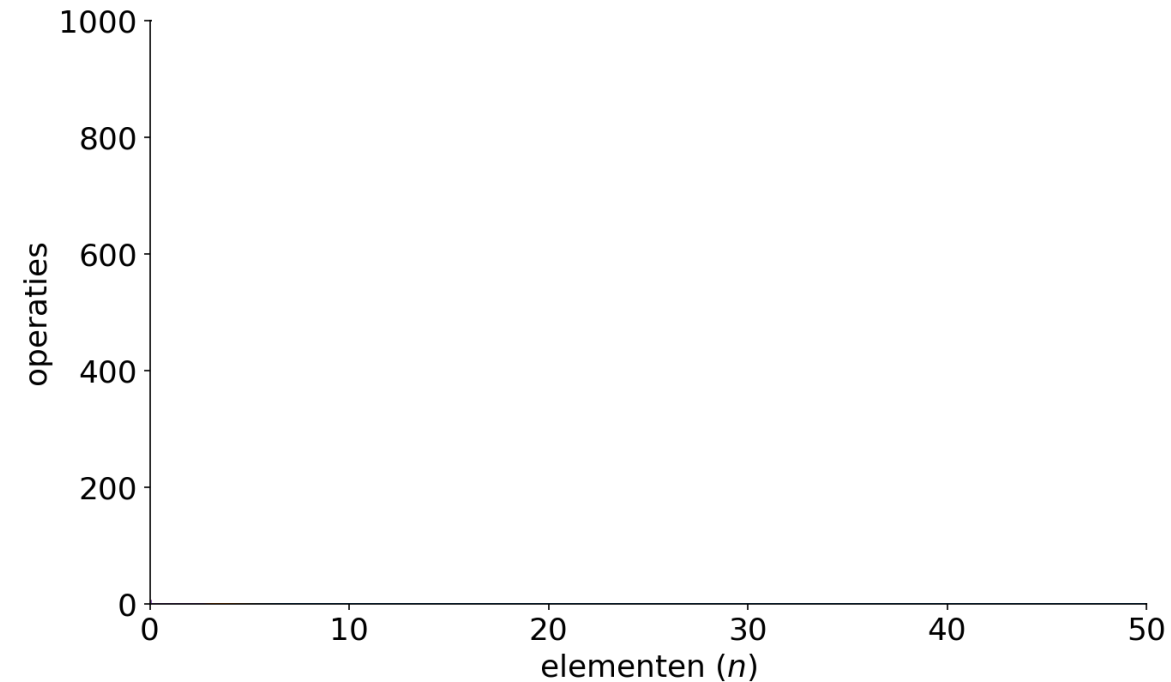
Handelsreizigersprobleem

Aantal steden	Aantal mogelijke routes	Rekentijd
2	1	1 ns
5	24	24 ns
10	362880	0,36 ms
13	479001600	0,5 s
15	87178291200	1,5 minuut
17	20922789888000	5,8 uren
19	6402373705728000	2,5 maanden
21	2432902008176640000	77 jaren
23	1124000727777607680000	356 eeuwen
25	620448401733239439360000	196742 eeuwen



Complexiteit – oefening

- Schets in één figuur met verschillende kleuren de grafieken van:
 - $f_1(n): 1$
 - $f_2(n): n$
 - $f_3(n): n^2$
 - $f_4(n): 2^n$
 - $f_5(n): n!$
- Laat de x-as lopen van $n = [0 \dots 50]$ en de y-as van $f(n) = [0 \dots 1000]$



Complexiteit van zoeken – oefening

- Gegeven een gesorteerde lijst van 13 namen:
 - Hoeveel stappen zijn er nodig om een specifieke naam te vinden bij *lineair* zoeken
 - in het beste geval?
 - in het slechtste geval?
 - Hoeveel stappen zijn er nodig om een specifieke naam te vinden bij *binair* zoeken
 - in het beste geval?
 - in het slechtste geval?

Complexiteit van zoeken – oefening

- Gegeven een gesorteerde lijst van 67 elementen:
 - Hoeveel stappen zijn er nodig bij *linear* zoeken om te concluderen dat een element *niet* in de lijst voorkomt?
 - Hoeveel stappen zijn er nodig bij *binair* zoeken om te concluderen dat een element *niet* in de lijst voorkomt?

Huiswerk

- Maak Oefening AI2.1: Lineair zoeken (zie Canvas):
 1. Retourneer of het gezochte item in de lijst staat
 2. Retourneer de index van het gezochte item
 3. Retourneer óók het aantal benodigde vergelijkingen
- Maak Oefening AI2.2: Binair zoeken (Zie Canvas)
 - (Gebruik hiervoor de pseudocode)
 1. Retourneer of het gezochte item in de lijst staat
 2. Retourneer de index van het gezochte item
 3. Retourneer óók het aantal benodigde vergelijkingen
- Werk aan Opdracht AI1: Algoritmiek